

NUMPY

What is Numpy?

NumPy (Numerical Python) is the foundational library for scientific computing in Python.

It provides a high-performance, multi-dimensional array object called ndarray, along with tools for working with these arrays efficiently.

mathematical calculations arrays mean median polynomial variance matrix algebra multi pixels game development

Why is Numpy?

Python lists are flexible but slow for large-scale numerical operations. NumPy arrays are: Faster — operations execute in compiled C code Memory efficient — stores homogeneous data types compactly Convenient — supports element-wise operations without loops Ecosystem foundation — used by Pandas, Matplotlib, Scikit-learn, etc.

Key Numpy Concepts:

Term	Meaning
ndarray	N-dimensional array — the core object in NumPy
shape	Tuple describing dimensions, e.g. (3, 4) means 3 rows, 4 cols
dtype	Data type of elements: int32, float64, bool, etc.
axis	Direction of operations: axis=0 (rows), axis=1 (cols)
broadcasting	Rules that allow operations on arrays of different shapes
vectorization	Applying operations to whole arrays without explicit loops

What is data visualization?

Working with Numpy

1.install numpy 2. import numpy 3. use numpy methods

```
# Installation (run in terminal / command prompt)

pip install numpy
```

```
File "/tmp/ipykernel_20763/3751068026.py", line 3
  pip install numpy
  ^
SyntaxError: invalid syntax
```

```
# Standard import convention (always use 'np' alias)

import numpy as np
```

```
# Check version
print(np.__version__)
```

```
2.0.2
```

Feature	Python List	NumPy Array
Element types	Any Python object	Fixed dtype only
Memory layout	Pointers → scattered heap objects	One contiguous block
Numeric speed	Python loops (slow)	Vectorized C code (fast)
Size flexibility	Dynamic (append / remove)	Fixed (mostly immutable)

list vs Array

Arrays

Type	Meaning
1D Array	Single row
2D Array	Rows and columns
3D Array	Multiple 2D arrays

Creating multiple types of arrays

```
# 1D Array it will be referred as vector
array1d=np.array([1,2,3])
print(array1d)
```

```
[1 2 3]
```

```
# 2D Array it will be referred as Matrix

array2d=np.array([[1,2,3],[4,5,6],[7,8,9]])
print(array2d)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
# 3D Array it will be referred as tensor
array3d=np.array([[[1,2,3],[4,5,6],[7,8,9]],[[10,11,12],[13,14,15],[16,17,18]]])
print(array3d)
```

```
[[[ 1  2  3]
   [ 4  5  6]
   [ 7  8  9]]

 [[10 11 12]
  [13 14 15]
  [16 17 18]]]
```

1D VS 2D VS 3D

```
# This is formatted as code
```

Built-in Array Creation Functions

```
# ZEROS AND ONES
np.zeros((2,5))
"""
np.zeros() function will create a matrix with values of 0 based
on given parameters
"""
```

Show hidden output

```
np.ones((2,5))
"""
np.ones() function will create a matrix with values of 1 based
on given parameters
"""
```

```
'\nnp.ones() function will create a matrix with values of 1 based \non given parameters\n'
```

```
np.full((2,5),5)
"""
np.full() function will create a matrix with values of 0 based
on given parameteres
"""
```

Show hidden output

```
#IDENTITY MATRIX
np.eye(4)
"""
np.eye() function will create a identity matrix with values of 0 and 1
"""
```

```
'\nnp.eye() function will create a identity matrix with values of 0 and 1\n'
```

```
#RANGES
np.arange(0,15,4)
"""
np.arange(start,stop,step)
"""
```

```
'\nnp.arrange(start,stop,step)\n'
```

```
np.linspace(2,10,12)
```

```
array([ 2.          ,  2.72727273,  3.45454545,  4.18181818,  4.90909091,
        5.63636364,  6.36363636,  7.09090909,  7.81818182,  8.54545455,
        9.27272727, 10.          ])
```

Random Arrays

```
# Uniform Random[0,1]
np.random.rand(9,3)
```

```
array([[0.52447175, 0.07206643, 0.6313266 ],
       [0.01947662, 0.99189993, 0.23052007],
       [0.39674086, 0.33783592, 0.2744089 ],
       [0.62449467, 0.78402448, 0.74065336],
       [0.54927633, 0.41510097, 0.25948586],
       [0.41172209, 0.75439829, 0.19982319],
       [0.36933579, 0.64074985, 0.60506222],
       [0.70375396, 0.49089473, 0.08861889],
       [0.49643608, 0.83920267, 0.10590679]])
```

```
# Standard Normal Distribution
np.random.randn(3,3)
```

```
array([[ 1.01963331, -0.13655436,  0.15688647],
       [-1.0752201 ,  0.4228149 , -0.54407495],
       [-1.5625948 ,  1.03956801, -2.34778154]])
```

```
# Random Integers
np.random.randint(1,5,size=(4,4))
```

```
array([[2, 1, 2, 3],
       [2, 4, 1, 3],
       [2, 3, 2, 3],
       [3, 1, 3, 2]])
```

```
# Uninitialized ( fast, but values are garbage)
np.empty((3,5))
```

```
array([[6.71482317e-316, 0.00000000e+000, 0.00000000e+000,
        4.79243676e-322, 7.85215132e-316],
       [6.76254005e-310, 1.50884601e-142, 8.42241414e+252,
        1.67388222e+296, 2.42092166e-322],
       [6.76254005e-310, 6.76254005e-310, 6.76254005e-310,
        6.76254005e-310, 6.32404027e-322]])
```

Array Attributes

```
arr= np.array([["a","b","c","d","e"],[1,2,3,4,5]])
print(arr.shape)      # - rows x columns
print(arr.ndim)       #      - number of dimensions
print(arr.size)       #      - total number of elements
print(arr.dtype)      # - data type
print(arr.itemsize)   #      - bytes per element
print(arr.nbytes)     #      - total bytes used
```

```
(2, 5)
2
10
<U21
84
840
```

Indexing and Slicing

```
# 1D INDEXING
a = np.array([10, 20, 30, 40, 50])

# Positive indexing (left to right, starts at 0)
print(a[0])    # 10
print(a[2])    # 30
print(a[-1])   # 50 (last element)

# Slicing: [start : stop : step]
print(a[1:4])  # [20 30 40]
print(a[:3])   # [10 20 30]
print(a[::2])  # [10 30 50]
print(a[::-1]) # [50 40 30 20 10] (reversed)
```

```
10
30
50
[20 30 40]
[10 20 30]
[10 30 50]
[50 40 30 20 10]
```

```
#2D INDEXING
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# Single element: [row, col]
print(m[0, 0])    # 1
print(m[1, 2])    # 6

# Row slicing
print(m[0, :])    # [1 2 3] - first row
# Column Slicing
print(m[:, 1])    # [2 5 8] - second column

# Sub-matrix
print(m[0:2, 1:3]) # [[2 3] [5 6]]
```

```
1
6
[1 2 3]
[2 5 8]
[[2 3]
 [5 6]]
```

```
# Boolean / Fancy Indexing
a = np.array([10, 25, 8, 42, 17])

# Boolean mask
mask = a > 15
print(mask)      # [False  True False  True  True]
print(a[mask])   # [25 42 17]

# Shorthand
print(a[a > 15]) # [25 42 17]

# Fancy indexing (select by index list)
print(a[[0, 3]]) # [10 42]
```

```
[False  True False  True  True]
[25 42 17]
```

```
[25 42 17]
[10 42]
```

ARRAY OPERATIONS

```
# Element-wise Arithmetic
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b) # [5 7 9]
print(a - b) # [-3 -3 -3]
print(a * b) # [ 4 10 18]
print(a / b) # [0.25 0.4 0.5]
print(a ** 2) # [1 4 9] - squaring
print(a + 10) # [11 12 13] - scalar broadcast
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4 0.5 ]
[1 4 9]
[11 12 13]
```

```
# Mathematical Functions
a = np.array([1.0, 4.0, 9.0, 16.0])

np.sqrt(a)      # [1. 2. 3. 4.]
np.exp(a)       # e^x for each element
np.log(a)       # natural log
np.log10(a)     # base-10 log
np.sin(a)       # sine
np.abs(a)       # absolute value
np.floor(a)     # round down
np.ceil(a)      # round up
np.round(a, 2)  # round to 2 decimal places
```

```
array([ 1.,  4.,  9., 16.])
```

```
# Aggregate / Reduction Functions
m = np.array([[1, 2, 3],
              [4, 5, 6]])

np.sum(m)        # 21 (total)
np.sum(m, axis=0) # [5 7 9] (column sums)
np.sum(m, axis=1) # [ 6 15] (row sums)

np.mean(m)       # 3.5
np.median(m)     # 3.5
np.std(m)        # standard deviation
np.var(m)        # variance

np.min(m)        # 1
np.max(m)        # 6
np.argmin(m)     # 0 (flat index of min)
np.argmax(m)     # 5 (flat index of max)
```

```
np.int64(5)
```

Reshaping and Restructuring

```
a = np.arange(12)      # [ 0  1  2  3  4  5  6  7  8  9 10 11]

# Reshape (total elements must match)
b = a.reshape(3, 4)    # 3x4 matrix
c = a.reshape(2, 2, 3) # 3D tensor
print(a)
print(b)
print(c)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[[ 0  1  2]
  [ 3  4  5]]

 [[ 6  7  8]
  [ 9 10 11]]]
```

```
# Flatten / Ravel
b.flatten()           # Returns new 1D copy
b.ravel()             # Returns 1D view (no copy if possible)

print(b)

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
# Transpose
b.T                  # Swap rows and columns
np.transpose(b)      # Same as b.T

array([[ 0,  4,  8],
       [ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11]])
```

Stacking and Splitting

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Stacking
np.vstack([a, b])  # [[1 2][3 4][5 6][7 8]] - vertical stack
np.hstack([a, b])  # [[1 2 5 6][3 4 7 8]] - horizontal stack
np.concatenate([a, b], axis=0) # same as vstack

# Splitting
arr = np.arange(12).reshape(4, 3)
np.vsplit(arr, 2)  # Split into 2 equal horizontal halves
np.hsplit(arr, 3)  # Split into 3 equal vertical halves
```

```
[array([[0],
       [3],
       [6],
       [9]]),
 array([[ 1],
       [ 4],
       [ 7],
       [10]]),
 array([[ 2],
       [ 5],
       [ 8],
       [11]])]
```

Double-click (or enter) to edit

Practice Problems

- Q1. Create a 1D array of even numbers from 2 to 20 (inclusive). hint: Use `np.max()` and `np.argmax()`
- Q2. Create a 5x5 identity matrix and print it. Use `.reshape()`
- Q3. Given `arr = np.array([3, 7, 1, 9, 2, 5, 8, 4])`, find the maximum value and its index without using Python's built-in `max()`. hint: Use `np.max()` and `np.argmax()`
- Q4. Reshape the array `np.arange(24)` into a 3D array of shape (2, 3, 4). hint: Use `.reshape()`
- Q5. Given `m = np.array([[4,9,2],[7,1,5],[3,8,6]])`, extract all elements greater than 5 using boolean indexing.) hint: Create a boolean mask first
- Q6. Create two 3x3 random matrices A and B, compute their dot product, and print the trace (sum of diagonal elements). hint: Use `np.random.rand()`, `np.dot()`, and `np.trace()`
- Q7. Normalize the array `arr = np.array([10, 20, 30, 40, 50])` so that values range from 0 to 1. hint: Formula: $(arr - arr.min()) / (arr.max() - arr.min())$
- Q8. Stack two arrays `a = np.array([1,2,3])` and `b = np.array([4,5,6])` vertically and horizontally. hint: Use `np.vstack()` and `np.hstack()`

Answer Following

What is the difference between `np.arange()` and `np.linspace()`?

What does the `.reshape(-1, 1)` operation do to a 1D array?

Explain the difference between a view and a copy in NumPy.

What happens when you perform `a + b` where `a.shape=(3,1)` and `b.shape=(1,4)`?

Why is `np.dot(A, B)` different from `A * B`?

What is the purpose of the `axis` parameter in `np.sum()`?